

Один агент, любая задача

Скиллы в Deep Agents

Сергей Тращенко

Python-разработчик GigaChain, Сбер



TELEGRAM



РЕПОЗИТОРИЙ

Немного о себе и команде

GigaChain / Сбер

Делаем open source для агентов:

- SDK под GigaChat API (`gigachat`)
- интеграционный пакет для LangChain (`langchain-gigachat`)
- прокси-сервер `GPT2GIGA`
- универсальный ИИ-агент `GigaAgent`

Сергей Траценков

Python-разработчик GigaChain, ИИ-энтузиаст

Интересуюсь LangChain, LangGraph, Deep Agents и агентными harness-подходами



TELEGRAM



РЕПОЗИТОРИЙ

План

1. От agent loop к harness

Почему одной модели и набора tools недостаточно для длинных специализированных задач.

2. Deep Agents

Что это за обвязка: LangGraph, файловая система, память, инструменты, middleware и наблюдаемость.

3. Skills

Как навык выбирается, как читается `SKILL.md`, где живут helper-скрипты и референсы.

4. Практика

Соберём `xlsx-skill` и используем его для Excel-отчёта с формулами.



TELEGRAM



РЕПОЗИТОРИЙ

Проблема

Как затюнить универсального агента под узкоспециализированные задачи – и не перегрузить контекст инструкциями?



TELEGRAM



РЕПОЗИТОРИЙ

Почему большой prompt не спасает

System prompt

Подходит для базового поведения агента.

Плохо масштабируется, если складывать туда все узкие процедуры и исключения.

Tools

Дают агенту действия.

Но не описывают процесс целиком; а когда tools много, их описания раздувают контекст и усложняют выбор.

Skills

Дают специализацию по задаче.

В prompt попадает короткий индекс, а подробности открываются только при необходимости.



TELEGRAM



РЕПОЗИТОРИЙ

ReAct-агент vs agent harness

Harness – это слой вокруг модели, который делает agent loop пригодным для длинных задач: даёт среду, состояние, планирование и правила работы с инструментами.

	Обычный ReAct-агент	Agent harness
Взаимодействие с миром	API-вызовы под конкретную задачу	файловая система, shell/code, внешние tools
Память	история текущего диалога	файлы, store, checkpointing между шагами
Инструменты	узкий набор tools под сценарий	базовый набор операций + подключаемые tools
План и контекст	держатся в сообщениях и легко теряются	выносятся в артефакты: todo, файлы, summary
Охват задач	агент под один класс задач	general-purpose агент для разных сценариев

Skills ложатся в эту обязанку как способ специализации: агент видит короткий индекс, а инструкции и helper-код читает из файлов по необходимости.



TELEGRAM



РЕПОЗИТОРИЙ

Стек: от langchain-core до Deep Agents

Deep Agents – готовый harness: planning · filesystem · subagents · skills

LangChain – agents, tools, middleware API

LangGraph – runtime графа, состояние, checkpointing, streaming

langchain-core – сообщения, модели, tools, Runnable



TELEGRAM



РЕПОЗИТОРИЙ

Deep Agents = открытая обвязка

Что даёт

- один general-purpose агент вместо отдельного агента под каждую задачу;
- преднастроенные механизмы для длинных сценариев;
- возможность менять компоненты обвязки, не меняя модель;
- поведение ближе к coding-agent системам вроде Claude Code / Codex CLI.

Что можно настраивать

- system prompt;
- tools;
- backend файловой системы;
- memory / store / checkpointer;
- middleware;
- subagents;
- **skills.**



TELEGRAM



РЕПОЗИТОРИЙ

Две формы обвязки

deepagents

Библиотека / SDK.

Встраиваем агента в свой Python-код, настраиваем модель, tools, backend, skills, memory.

Сегодня работаем здесь.

deepagents-code

Готовый coding-agent в терминале.

Реализован на JS/TS версии SDK: готовый терминальный coding-agent поверх Deep Agents.

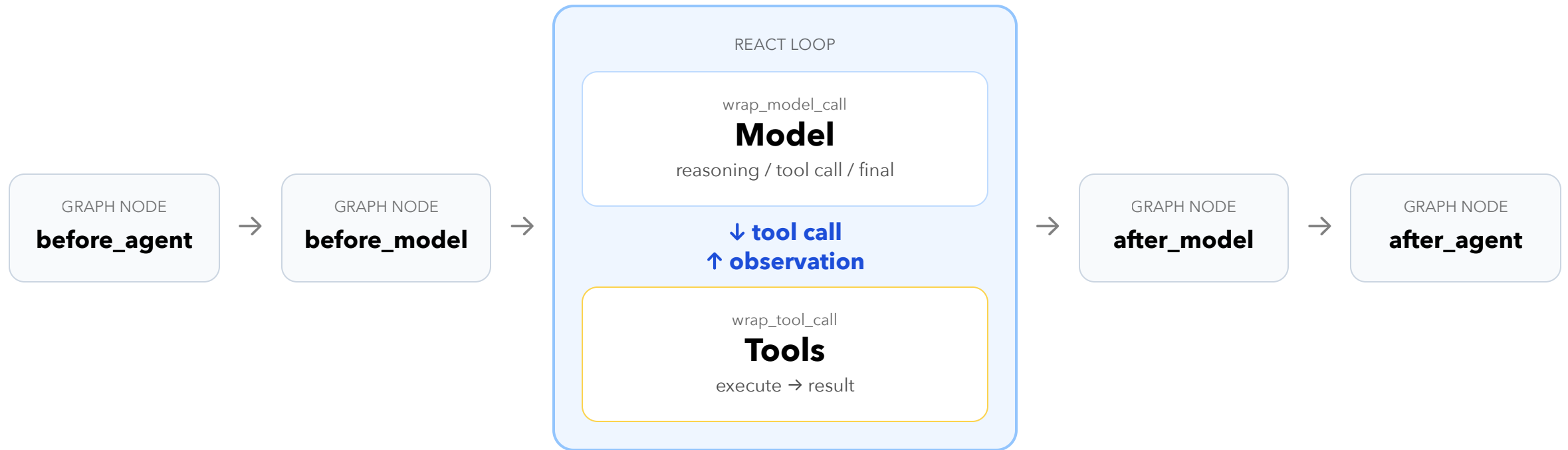


TELEGRAM



РЕПОЗИТОРИЙ

Внутри агента: ReAct-цикл в графе



before_* / after_*

Отдельные ноды LangGraph: вставляются в поток выполнения агента.



TELEGRAM

wrap_model_call / wrap_tool_call

Обёртки вызова: перехватывают model-call или tool-call внутри ReAct-цикла.



РЕПОЗИТОРИЙ

Skills: специализация по требованию

Skill – это навык в файловой системе агента: короткое описание видно сразу, подробности открываются только после выбора.

До выбора

Агент видит только короткий индекс:

name + description

Этого достаточно, чтобы понять, подходит ли навык к задаче.

После выбора

Агент открывает детали навыка:

SKILL.md

scripts

references

examples

задача



выбор skill



чтение инструкции



выполнение



проверка



TELEGRAM



РЕПОЗИТОРИЙ

Из чего состоит skill

SKILL.md

- когда использовать навык;
- какой порядок действий;
- какие ограничения помнить;
- как понять, что результат готов.

scripts/

- генерация файлов;
- парсинг данных;
- вызов библиотек;
- валидация результата.

references/

- шаблоны;
- примеры;
- правила оформления;
- дополнительные материалы.

Хороший skill переносит повторяемую механику из рассуждения модели в инструкцию и код.



TELEGRAM



РЕПОЗИТОРИЙ

Что внутри SKILL.md

```
---  
name: xlsx  
description: Когда выбирать этот навык  
---  
  
# Назначение  
Что агент должен уметь делать.  
  
# Порядок работы  
1. Что прочитать или уточнить.  
2. Какие действия выполнить.  
3. Какие scripts использовать.  
  
# Ограничения  
Поддерживаемые форматы, запреты, важные договорённости.  
  
# Проверка результата  
Как агент понимает, что задача выполнена корректно.
```

`description` помогает выбрать навык, тело `SKILL.md` ведёт агента по процессу после выбора.



TELEGRAM



РЕПОЗИТОРИЙ

Skills, tools и memory

	Tools	Skills	Memory
Отвечают на вопрос	Что агент может вызвать?	Как выполнить специализированную задачу?	Что агент должен помнить?
Что внутри	Функция/API и её описание	Инструкция, scripts, references	Факты, предпочтения, правила проекта
Когда полезно	Нужно действие	Нужен процесс	Нужен устойчивый контекст
Пример	<code>write_file</code> , <code>execute</code> , <code>read_csv</code>	«Собрать Excel-отчёт с формулами»	«В этой команде отчёт называется sales_report»

Tools дают действия, **memory** хранит контекст, **skills** задают процедуру.



TELEGRAM



РЕПОЗИТОРИЙ

Практика

Реализуем xlsx-skill и делаем Excel-отчёт с формулами



TELEGRAM



РЕПОЗИТОРИЙ

Почему не берём готовый `xlsx-skill`

Источник: репозиторий `anthropics/skills`. Берём его как reference, потому что Anthropic фактически сформулировали pattern skills для агентов.

`skill-creator`

Лицензия: **Apache 2.0**.

Его можно подключить к Deep Agents и использовать как инструмент для создания своих навыков.

`xlsx`, `docx`, `pdf`, `pptx`

Лицензия: **proprietary**.

Их нельзя просто вынести из сервисов Anthropic, скопировать, создать производную версию или распространять.

Поэтому процесс такой: открытый `skill-creator` используем, закрытый `xlsx` изучаем как идею, а свой навык пишем заново.



TELEGRAM



ANTHROPICS/SKILLS



РЕПОЗИТОРИЙ

Что будем собирать: свой `xlsx-skill`

Не копия Anthropic skill, а новый навык для Deep Agents: написанный своими словами и со своими helper-скриптами.

```
skills/  
└─ xlsx/  
    └─ SKILL.md  
        └─ scripts/  
            └─ create_workbook.py  
            └─ validate_workbook.py
```

Что берём из reference

Идею устройства навыка: когда он триггерится, как разбивать работу на инструкцию и scripts, какие проверки нужны.

Что пишем сами

`SKILL.md` и Python-код на `openpyxl` : таблица, формулы `SUM` / `AVERAGE` , проверка результата.



TELEGRAM



РЕПОЗИТОРИЙ

Workflow в ноутбуке

1. Клонировем `anthropics/skills` во временную папку
2. Копируем в проект только `skill-creator` – он Apache 2.0
3. Просим агента изучить proprietary `xlsx` как reference: `SKILL.md` и scripts
4. Через `skill-creator` создаём свой `skills/xlsx/` с нуля
5. Проверяем навык на `output/test.xlsx` с формулой `SUM`

Практика проходит полный цикл: изучаем reference, создаём свой skill, применяем его и проверяем результат.



TELEGRAM



РЕПОЗИТОРИЙ

Задача для xlsx-skill

Сгенерировать Excel-отчёт по продажам.

Данные

- регионы;
- товары;
- количество;
- цена;
- сумма строки.

Формулы и проверка

- итоговая сумма через `SUM` ;
- среднее значение через `AVERAGE` ;
- базовое форматирование;
- проверка, что формулы записаны в ячейки.

Проверку запускаем в новой сессии: индекс `skills` пересобирается, и новый ``xlsx`` появляется в системном `prompt`.



TELEGRAM



РЕПОЗИТОРИЙ

Наблюдаемость: Phoenix

В ноутбуке мы запускаем локальный Phoenix и подключаем OpenInference-инструментирование LangChain. Так видно не только финальный ответ, но и шаги агента.

В трейсе видно

- какие сообщения ушли в модель;
- какие tools вызвал агент;
- прочитал ли он `SKILL.md` ;
- какие файлы и scripts использовал;
- расход токенов и время выполнения;
- где появились лишние шаги или цикл.

Зачем это в практике

- отлаживать поведение агента по шагам;
- проверять, что новый skill реально сработал;
- собирать удачные и неудачные прогоны в датасет;
- запускать эксперименты и оценки после изменений;
- улучшать `SKILL.md` и helper-скрипты по trace.



TELEGRAM



РЕПОЗИТОРИЙ

Материалы

Репозиторий, ноутбук, QR-коды



TELEGRAM



РЕПОЗИТОРИЙ

Скачать репозиторий

Отсканируйте QR-код «репозиторий» или откройте ветку с материалами вебинара.

```
git clone --branch codex/deep-agents-only-materials \  
https://github.com/trashchenkov/agent-skills-masterclass  
cd agent-skills-masterclass
```

Ветка отдельная, чтобы материалы вебинара не смешивались с прошлой конференционной версией.



TELEGRAM



РЕПОЗИТОРИЙ

Установить окружение

Нужны Python 3.12+ и `uv`.

```
uv init . --python 3.12 --no-workspace

uv add jupyter \
  langchain langchain-core langchain-openrouter \
  deepagents langgraph \
  arize-phoenix openinference-instrumentation-langchain \
  opentelemetry-sdk opentelemetry-exporter-otlp \
  openpyxl python-dotenv
```

Для практики с Excel ключевые зависимости: `deepagents`, `langgraph`, `openpyxl`, Phoenix-инструментирование.



TELEGRAM



РЕПОЗИТОРИЙ

Запустить ноутбук

Добавьте ключ модели в `.env` и запустите Jupyter через `uv` :

```
echo "OPENROUTER_API_KEY=sk-or-v1- ..." >> .env
uv run jupyter notebook masterclass.ipynb
```

Phoenix откроется локально: `http://localhost:6006`

Там будем смотреть, как агент выбирает skill, читает инструкции, запускает tools и проверяет результат.



TELEGRAM



РЕПОЗИТОРИЙ

Переходим к ноутбуку

Делаем xlsx-skill

QR-коды ведут в ветку репозитория с материалами вебинара и в Telegram-канал.



TELEGRAM



РЕПОЗИТОРИЙ